



Known Streaks Issue

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!

Protected Files

The use of file encryption is often still lacking in **private** and **business** matters. Even today, emails containing job applications, account statements, or contracts are often sent unencrypted. This is grossly negligent and, in many cases, even punishable by law. For example, GDPR demands the requirement for encrypted storage and transmission of personal data in the European Union. Especially in business cases, this is quite different for emails. Nowadays, it is pretty common to communicate **confidential** topics or send **sensitive** data **by email**. However, emails are not much more secure than postcards, which can be intercepted if the attacker is positioned correctly.

More and more companies are increasing their IT security precautions and infrastructure through training courses and security awareness seminars. As a result, it is becoming increasingly common for company employees to encrypt/encode sensitive files. Nevertheless, even these can be cracked and read with the right choice of lists and tools. In many cases, **symmetric encryption** like **AES-256** is used to securely store individual files or folders. Here, the **same key** is used to encrypt and decrypt a file.

Therefore, for sending files, **asymmetric encryption** is used, in which **two separate keys** are required. The sender encrypts the file with the **public key** of the recipient. The recipient, in turn, can then decrypt the file using a **private key**.

Hunting for Encoded Files

Many different file extensions can identify these types of encrypted/encoded files. For example, a useful list can be found on [FileInfo](#). However, for our example, we will only look at the most common files like the following:

Hunting for Files

Protected Files

```
cry0l1t3@unixclient:~$ for ext in $(echo ".xls .xls* .xltx .csv .od* .doc .doc* .pdf .pot .pot* .pp*");do echo -e
File extension:  .xls
File extension:  .xls*
File extension:  .xltx
File extension:  .csv
/home/cry0l1t3/Docs/client-emails.csv
/home/cry0l1t3/ruby-2.7.3/gems/test-unit-3.3.4/test/fixtures/header-label.csv
/home/cry0l1t3/ruby-2.7.3/gems/test-unit-3.3.4/test/fixtures/header.csv
/home/cry0l1t3/ruby-2.7.3/gems/test-unit-3.3.4/test/fixtures/no-header.csv
/home/cry0l1t3/ruby-2.7.3/gems/test-unit-3.3.4/test/fixtures/plus.csv
/home/cry0l1t3/ruby-2.7.3/test/win32ole/orig_data.csv
File extension:  .od*
/home/cry0l1t3/Docs/document-temp.odt
/home/cry0l1t3/Docs/product-improvements.odp
/home/cry0l1t3/Docs/mgmt-spreadsheet.ods
...SNIP...
```

If we encounter file extensions on the system that we are not familiar with, we can use the search engines that we are familiar with to find out the technology behind them. After all, there are hundreds of different file extensions, and no one is expected to know all of them by heart. First, however, we should know how to find the relevant information that will help us. Again, we can use the steps we already



Known Streaks Issue

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!

Protected Files
<pre>cry0l1t3@unixclient:~\$ grep -rnw "PRIVATE KEY" /* 2>/dev/null grep ":1" /home/cry0l1t3/.ssh/internal_db:1:-----BEGIN OPENSsh PRIVATE KEY----- /home/cry0l1t3/.ssh/SSH.private:1:-----BEGIN OPENSsh PRIVATE KEY----- /home/cry0l1t3/Mgmt/ceil.key:1:-----BEGIN OPENSsh PRIVATE KEY-----</pre>

Most SSH keys we will find nowadays are encrypted. We can recognize this by the header of the SSH key because this shows the encryption method in use.

Encrypted SSH Keys

Protected Files
<pre>cry0l1t3@unixclient:~\$ cat /home/cry0l1t3/.ssh/SSH.private -----BEGIN RSA PRIVATE KEY----- Proc-Type: 4,ENCRYPTED DEK-Info: AES-128-CBC,2109D25CC91F8DBFCEB0F7589066B2CC 8Uboy0afrTahejVGmB7kgvxkqJL0czb1I0/hEzPU1leCqhCKBlxYldM2s65jhflD 4/0H4ENhU7qpJ62KlrnZhFX8UwYBmebNDvG12oE7i21hB/9UqZmmHktjD3+0YTsD ...SNIP...</pre>

If we see such a header in an SSH key, we will, in most cases, not be able to use it immediately without further action. This is because encrypted SSH keys are protected with a passphrase that must be entered before use. However, many are often careless in the password selection and its complexity because SSH is considered a secure protocol, and many do not know that even lightweight [AES-128-CBC](#) can be cracked.

Cracking with John

[John The Ripper](#) has many different scripts to generate hashes from files that we can then use for cracking. We can find these scripts on our system using the following command.

John Hashing Scripts

Protected Files
<pre>dbcypth0n@htb[/htb]\$ locate *2john* /usr/bin/bitlocker2john /usr/bin/dmg2john /usr/bin/gpg2john /usr/bin/hccap2john /usr/bin/keepass2john /usr/bin/putty2john /usr/bin/racf2john /usr/bin/rar2john /usr/bin/uaf2john /usr/bin/vncpcap2john /usr/bin/wlanhcx2john /usr/bin/wpapcap2john /usr/bin/zip2john /usr/share/john/1password2john.py /usr/share/john/7z2john.pl /usr/share/john/DPAPImk2john.py</pre>



Known Streaks Issue

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!

```
/usr/share/john/androidbackup2john.py
...SNIP...
```

We can convert many different formats into single hashes and try to crack the passwords with this. Then, we can open, read, and use the file if we succeed. There is a Python script called `ssh2john.py` for SSH keys, which generates the corresponding hashes for encrypted SSH keys, which we can then store in files.

Protected Files

```
dbcyp0n@htb[/htb]$ ssh2john.py SSH.private > ssh.hash
dbcyp0n@htb[/htb]$ cat ssh.hash

ssh.private:$sshng$0$8$1C258238FD2D6EB0$2352$f7b...SNIP...
```

Next, we need to customize the commands accordingly with the password list and specify our file with the hashes as the target to be cracked. After that, we can display the cracked hashes by specifying the hash file and using the `--show` option.

Cracking SSH Keys

Protected Files

```
dbcyp0n@htb[/htb]$ john --wordlist=rockyou.txt ssh.hash

Using default input encoding: UTF-8
Loaded 1 password hash (SSH [RSA/DSA/EC/OPENSSH (SSH private keys) 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Note: This format may emit false positives, so it will keep trying even after
finding a possible candidate.
Press 'q' or Ctrl-C to abort, almost any other key for status
1234          (SSH.private)
1g 0:00:00:00 DONE (2022-02-08 03:03) 16.66g/s 1747Kp/s 1747Kc/s 1747KC/s Knightsing..Babying
Session completed
```

Protected Files

```
dbcyp0n@htb[/htb]$ john ssh.hash --show

SSH.private:1234

1 password hash cracked, 0 left
```

Cracking Documents

In the course of our career, we will come across many different documents, which are also password-protected to prevent access by unauthorized persons. Today, most people use Office and PDF files to exchange business information and data.

Pretty much all reports, documentation, and information sheets can be found in the form of Office DOCs and PDFs. This is because they offer the best visual representation of information. John provides a Python script called `office2john.py` to extract hashes from all common Office documents that can then be fed into John or Hashcat for offline cracking. The procedure to crack them remains the same.

Cracking Microsoft Office Documents

**Known Streaks Issue**

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!

```
dbcyp0n@htb[/htb]$ cat protected-docx.hash
```

```
Protected.docx:$office$*2007*20*128*16*7240...SNIP...8a69cf1*98242f4da37d916305d8e2821360773b7edc481b
```

Protected Files

```
dbcyp0n@htb[/htb]$ john --wordlist=rockyou.txt protected-docx.hash
```

```
Loaded 1 password hash (Office, 2007/2010/2013 [SHA1 256/256 AVX2 8x / SHA512 256/256 AVX2 4x AES])
Cost 1 (MS Office version) is 2007 for all loaded hashes
Cost 2 (iteration count) is 50000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
1234 (Protected.docx)
1g 0:00:00:00 DONE (2022-02-08 01:25) 2.083g/s 2266p/s 2266c/s 2266C/s trisha..heart
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Protected Files

```
dbcyp0n@htb[/htb]$ john protected-docx.hash --show
```

```
Protected.docx:1234
```

Cracking PDFs

Protected Files

```
dbcyp0n@htb[/htb]$ pdf2john.py PDF.pdf > pdf.hash
dbcyp0n@htb[/htb]$ cat pdf.hash
```

```
PDF.pdf:$pdf$2*3*128*-1028*1*16*7e88...SNIP...bd2*32*a72092...SNIP...0000*32*c48f001fdc79a030d718df5dbbdaad81d1f6f
```

Protected Files

```
dbcyp0n@htb[/htb]$ john --wordlist=rockyou.txt pdf.hash
```

```
Using default input encoding: UTF-8
Loaded 1 password hash (PDF [MD5 SHA2 RC4/AES 32/64])
Cost 1 (revision) is 3 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
1234 (PDF.pdf)
1g 0:00:00:00 DONE (2022-02-08 02:16) 25.00g/s 27200p/s 27200c/s 27200C/s bulldogs..heart
Use the "--show --format=PDF" options to display all of the cracked passwords reliably
Session completed
```

Protected Files

```
dbcyp0n@htb[/htb]$ john pdf.hash --show
```

```
PDF.pdf:1234
```

```
1 password hash cracked, 0 left
```

One of the major difficulties in this process is the generation and mutation of password lists. This is the prerequisite for successfully cracking the passwords for all recovered protected files and access points. This is because it is often no longer sufficient to use a known



Known Streaks Issue

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!

ended to select a longer, randomly generated password or a passphrase. Nevertheless, it is always worth attempting to crack passwords protected documents as they may yield sensitive data that could be useful to further our access.

VPN Servers

 Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

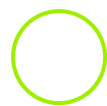
EU Academy 3

Medium Load 

PROTOCOL

 UDP 1337  TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

171ms 

Terminate Pwnbox to switch location

Start Instance

 / 1 spawns left

Waiting to start...



Known Streaks Issue

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!



Download VPN Connection
File

Target(s): [Click here to spawn the target system!](#)

+ 0 Use the cracked password of the user Kira and log in to the host and crack the "id_rsa" SSH key. Then, submit the password for the SSH key as the answer.

Submit your answer here...

+10 Streak pts



Submit



Show Solution

← Previous

Next →



Cheat Sheet



Resources



Go to Questions

Table of Contents

Introduction

Theory of Protection	✓
Credential Storage	✓
John The Ripper	✓

Remote Password Attacks

Network Services	✓
Password Mutations	✓
Password Reuse / Default Passwords	✓

Windows Local Password Attacks

Attacking SAM	✓
Attacking LSASS	✓
Attacking Active Directory & NTDS.dit	✓
Credential Hunting in Windows	✓

Linux Local Password Attacks

Credential Hunting in Linux	✓
Passwd, Shadow & Opasswd	✓

Windows Lateral Movement

Pass the Hash (PtH)	✓
Pass the Ticket (PtT) from Windows	✓
Pass the Ticket (PtT) from Linux	✓



Known Streaks Issue

We're fixing a streaks issue this week. Your streaks are safe. Thanks for your patience!

Password Management

Password Policies

Password Managers

Skills Assessment

 Password Attacks Lab - Easy

 Password Attacks Lab - Medium

 Password Attacks Lab - Hard

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left